

Drone de surface à voile

Rapport technique — Développement du firmware et de l'électronique

Plateforme :	LilyGO T-Beam V1.1 (ESP32 + SX1276 + GPS u-blox + AXP192)
Langage :	Arduino / C++, cible ESP32 Arduino Core 3.x
Auteur (code original) :	Mohamed EL JILY
Auteur (refonte) :	Facundo Arito
Version :	2.0 — May 24, 2026

Résumé

Ce document retrace l'intégralité du développement informatique et électronique du drone de surface à voile basé sur le module T-Beam d'Espressif/LilyGO. Il couvre le code original et ses limitations, la réécriture complète de l'architecture logicielle, les deux révisions du PCB, tous les problèmes matériels et logiciels rencontrés, ainsi que les solutions mises en place. Le document est structuré pour être mis à jour facilement au fil des phases futures du projet.

Projet de recherche — Drone autonome de surveillance océanographique

Table des matières

1. Présentation du projet

1.1. Concept physique

Le drone est un véhicule de surface propulsé principalement par la voile. Sa conception est originale : la voile est à profil d'aile et tourne librement autour d'un axe vertical. L'aérodynamique passive oriente automatiquement le profil à environ 45° de l'axe du drone par rapport au vent. Deux actionneurs principaux permettent de naviguer :

- **Servo aileron (voile) — Futaba S3003** : contrôle un petit volet sur le bord de fuite de la voile. Sa course utile est strictement de $\pm 10^\circ$. La déflexion détermine dans quel sens la voile propulse le drone, mais n'est pas continue : seules les deux positions extrêmes ont une signification physique (babord ou tribord).
- **Servo safran/rotor — Graupner Regatta ECO II (mod. 5176)** : contrôle le différentiel entre la rotation libre de la voile et un rotor fixe à la poupe. C'est le principal actionneur de cap (lacet). C'est un *treuil de voile multi-tour positonnel* : le signal PWM commande une position cible de l'arbre (et non une vitesse). Centre = $1500\ \mu\text{s}$, course complète = 6 tours, maintien de position sous charge.
- **Propulseurs (ESC)** : utilisés uniquement en l'absence de vent suffisant. Permettent la propulsion et la direction par poussée différentielle.

Sans capteur de vent, la direction du vent est **déduite** du track GPS et de l'état des actionneurs. L'estimation nécessite un déplacement minimal de 30 m et deux amures (bords) observées.

1.2. Matériel principal

Composant	Description
LilyGO T-Beam V1.1	Module intégrant ESP32 WROVER, LoRa SX1276 433 MHz, GPS u-blox NEO-6M, AXP192 (gestion d'énergie), porte-batterie 18650
Futaba S3003	Servo aileron voile, centre à $1520\ \mu\text{s}$, $\pm 10^\circ$
Graupner Regatta ECO II	Treuil safran multi-tour, $1500\ \mu\text{s}$ = centre, $1000\text{--}2000\ \mu\text{s}$ = plage, jusqu'à 3,3 A en charge

Composant	Description
Pro-Tronik PTR-6A	Émetteur 6 voies FHSS 2,4 GHz, PWM 1000–2000 μ s
Pro-Tronik R8X	Récepteur 8 voies, PWM standard 50 Hz
Pro-Tronik Black Fet	ESC série, programmation par carte EPRG-3
Antenne GPS	Taoglas ADFGP.25A, active, connecteur u.FL
PCB v1 / PCB v2	Carte d'interface custom (KiCad 9)

2. Code original — `boat.ino`

2.1. Architecture et fonctionnement

Le code original (`Informatica/Arduino/boat/boat.ino`), écrit par Mohamed EL JILY, constitue le point de départ du projet. Il est organisé en modules :

- `Gps.cpp/.hpp` — lecture GPS (Serial1, TinyGPSPlus), historique de positions, lissage du cap
- `ServoControl.cpp/.hpp` — contrôle servo via ESP32Servo
- `RadioReceiver.cpp/.hpp` — lecture PWM par interruptions GPIO (3 canaux : PWM1, PWM2, SEL)
- `MotorControl.cpp/.hpp` — armement et contrôle ESC
- `boat.ino` — boucle principale, logique de navigation à la voile (observation du vent, waypoints, lofer/abattre/virement de bord)
- `config_pins.h` — correspondance GPIO

La logique de navigation est implémentée : calcul de l'angle relatif vent/cap, changement d'amure automatique (VDB), lofer/abattre. Les waypoints sont reçus par LoRa depuis un serveur en JSON.

2.2. Problèmes et limitations identifiés

Lors de la reprise du projet, plusieurs problèmes bloquants ont été identifiés :

2.2.1. Bibliothèque ESP32Servo — conflit LEDC/interruptions

Listing 1: ServoControl.cpp original – utilisation d’ESP32Servo

```

1 void ServoControl::init() {
2     sailServo.setPeriodHertz(SERVO_PWM_FREQ);
3     sailServo.attach(SERVO_SAIL_PIN, SERVO_PULSE_MIN,
4                     SERVO_PULSE_MAX);
5     ...
6 }
7 void ServoControl::setSailAngle(float angle) {
8     angle += 90;           // decalage vers 0-180deg
9     sailServo.write(constrain((int)angle, 0, 180));
10 }

```

ESP32Servo utilise le périphérique **LEDC** de l’ESP32. LEDC est incompatible avec le décodage PWM par interruptions GPIO : lorsque LEDC est actif, les interruptions RC sont perturbées et les valeurs lues deviennent aléatoires. Ce conflit rend impossible le contrôle simultané des servos et la lecture du récepteur radio.

2.2.2. Attribution des broches RC — conflit I2C/AXP192

Listing 2: config_pins.h original – broches RC sur I2C

```

1 #define RADIO_PWM1_IN 21 // GPIO 21 -- CH4 Aileron (= I2C SDA
   AXP192!)
2 #define RADIO_PWM2_IN 22 // GPIO 22 -- CH2 Safran (= I2C SCL
   AXP192!)
3 #define RADIO_SEL_IN 23 // GPIO 23 -- CH6 Select. (= LoRa RST
   hardware!)

```

GPIO21 et GPIO22 sont les broches I2C internes du T-Beam utilisées par l’AXP192 (gestionnaire d’énergie). Les utiliser comme entrées RC sans couper proprement l’I2C provoque des instabilités sur le bus. De plus, GPIO23 est connecté en interne au RESET du SX1276 (LoRa) sur le T-Beam V1.1 : attacher une interruption sur ce pin peut provoquer des resets intempestifs du module radio.

2.2.3. Broche batterie GPIO35 — drain de 20 mA continu

Listing 3: config_pins.h original – broche batterie

```

1 // GPIO35 (Battery ADC) - Voltage divider R~370 Ohm total
2 // Ne JAMAIS lire GPIO35 : drain 20 mA permanent depuis la batterie !

```

Le T-Beam V1.1 possède un diviseur résistif de *faible impédance* ($\approx 370 \Omega$ total) soudé en permanence sur GPI035. Même sans lire l'ADC, ce diviseur consomme ≈ 20 mA en continu, soit environ 1,8 W ! Le code original utilisait GPI015 à la place, ce qui est correct, mais sans documenter clairement ce problème.

2.2.4. Broche LoRa RST — erreur de documentation

Listing 4: config_pins.h original – LoRa RST incorrect

```
1 #define LORA_RST 14          // GPIO 14 (INCORRECT -- hardware = GPIO
    23)
```

Le hardware T-Beam V1.1 connecte le RESET du SX1276 à GPI023, pas à GPI014 comme documenté dans les exemples de LilyGO. Utiliser GPI014 ne déclenche aucun reset matériel, mais peut causer des comportements inattendus si ce pin est utilisé à d'autres fins.

2.2.5. Gestion ESC — absence d'armement et de limitation de variation

Le code original ne met pas en œuvre de procédure d'armement ESC : le moteur peut démarrer immédiatement si le signal de commande est trop élevé au démarrage. De même, aucune limitation de taux de variation (*slew rate*) n'est implémentée, ce qui peut provoquer des à-coups mécaniques lors de changements de consigne brusques.

3. PCB Version 1 — Carte d'interface originale

3.1. Description

La première carte PCB (fichier PCB_Electronica.kicad_pcb) réalise l'interface entre le T-Beam et les actionneurs. Elle comprend :

- Des connecteurs 3 broches pour les servos/ESC (PWM, +5V, GND)
- Des connecteurs pour le récepteur RC
- Un régulateur abaisseur 5 V (Pololu 2866) alimenté par la batterie
- Des bornes à vis pour la batterie et les sorties de puissance

3.2. Problèmes identifiés sur le PCB V1

Problème	Impact
Signaux PWM 3,3 V sans amplification niveau	Les servos et ESC (logique 5 V) peuvent ne pas détecter correctement les impulsions
Broches RC sur GPIO21/22	Conflict I2C/AXP192, impossible d'utiliser l'I2C après l'init sans couper le bus
ESC sur GPIO32/33	GPIO32/33 = LoRa DIO1/DIO2 sur le T-Beam ; non bloquant en phase 1-2 mais à surveiller
GPIO35 pour batterie	Diviseur $370\ \Omega = 20\ \text{mA}$ continu, non utilisable
Pas de protection en sens inverse	Risque en cas d'alimentation incorrecte

4. Réécriture logicielle — Architecture Post-Claude

4.1. Motivations

La réécriture complète du firmware vise à corriger toutes les limitations identifiées et à préparer l'architecture pour les phases autonomes futures. Les objectifs sont :

1. Utiliser MCPWM au lieu de LEDC pour les sorties actionneurs
2. Décodage RC entièrement par interruptions avec protection ISR
3. Architecture OOP en couches, extensible par phases
4. Armement ESC et limitation de variation (*slew rate*)
5. Gestion propre de l'énergie via AXP192
6. GPS, LoRa et batterie fonctionnant simultanément

4.2. Architecture en couches

Listing 5: Arborescence du projet Post-Claude

```
Informatica/Post-Claude/
  main/                                <- Sketch Arduino production
    main.ino
    src/
```

```

app/DroneApp.cpp      <- Orchestrateur principal (scheduler)
config/BoardConfig.h  <- Affectation des GPIO
config/Calibration.h  <- Valeurs uss servo/ESC
core/Types.h          <- Types partagés (RcFrame,
                        ActuatorCommand...)
control/ModeManager   <- CH5 -> mode de vol
control/ManualController <- Contrôle manuel RC
drivers/RcReceiver     <- Lecture PWM par interruptions
drivers/McpwmActuators <- Sorties MCPWM
drivers/AxpPower       <- Init AXP192
drivers/BatteryAdc     <- ADC tension batterie
drivers/GpsUart        <- Lecture GPS (TinyGPSPlus)
drivers/LoRaRadio      <- Driver SX1276
comm/LoRaComm         <- Protocole JSON LoRa
navigation/           <- Navigator, MissionPlan, MissionManager
PCBv3/                <- Sketch pour test sur platine d'essai

```

4.3. Décisions techniques fondamentales

4.3.1. MCPWM obligatoire — jamais LEDC

Listing 6: McpwmActuators.cpp – initialisation MCPWM

```

1 bool McpwmActuators::begin() {
2     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0A,
3                     BoardConfig::SAIL_SERVO_PIN);
4     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM0B,
5                     BoardConfig::ROTOR_SERVO_PIN);
6     mcpwm_gpio_init(MCPWM_UNIT_0, MCPWM1A, BoardConfig::ESC1_PIN);
7     mcpwm_config_t cfg = {};
8     cfg.frequency      = Calibration::PWM_FREQ_HZ;    // 50 Hz
9     cfg.duty_mode      = MCPWM_DUTY_MODE_0;          // actif haut
10    cfg.counter_mode    = MCPWM_UP_COUNTER;
11    mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_0, &cfg);
12    mcpwm_init(MCPWM_UNIT_0, MCPWM_TIMER_1, &cfg);
13    ...
14 }

```

Le périphérique MCPWM de l'ESP32 est indépendant des interruptions GPIO : les deux peuvent coexister sans perturbation mutuelle. Cette contrainte est **non négociable** pour ce projet.

4.3.2. Interruptions RC avec protection portMUX

Listing 7: RcReceiver.cpp – ISR et section critique

```

1 void IRAM_ATTR RcReceiver::handleEdge(uint8_t pin, Ch ch) {

```



```

2   const uint32_t now = micros();
3   const int      idx = static_cast<int>(ch);
4
5   if (gpio_get_level(static_cast<gpio_num_t>(pin))) {
6       portENTER_CRITICAL_ISR(&mux_);
7       riseUs_[idx] = now;
8       portEXIT_CRITICAL_ISR(&mux_);
9   } else {
10      uint32_t rise;
11      portENTER_CRITICAL_ISR(&mux_);
12      rise = riseUs_[idx];
13      portEXIT_CRITICAL_ISR(&mux_);
14      const uint32_t width = now - rise;
15      if (width >= RC_VALID_MIN_US && width <= RC_VALID_MAX_US) {
16          portENTER_CRITICAL_ISR(&mux_);
17          pulseUs_[idx] = static_cast<uint16_t>(width);
18          lastValidUs_[idx] = now;
19          portEXIT_CRITICAL_ISR(&mux_);
20      }
21  }
22 }

```

Les routines ISR sont placées en RAM (`IRAM_ATTR`) pour éviter les délais de lecture depuis le flash. La synchronisation avec la boucle principale utilise `portMUX`, mécanisme FreeRTOS approprié pour les ESP32 dual-core.

4.3.3. Toutes les consignes actionneurs en microsecondes

Tous les actionneurs sont commandés exclusivement en microsecondes (μs), jamais en degrés. Cela évite les conversions d'unités sources d'erreurs et correspond directement au protocole PWM des servos et ESC.

4.3.4. Slew rate — protection contre les à-coups

Listing 8: `McpwmActuators.cpp` – limitation de variation

```

1  uint16_t McpwmActuators::slew(uint16_t current, uint16_t target,
2                                uint16_t step) {
3      if (current < target) {
4          uint32_t next = static_cast<uint32_t>(current) + step;
5          return (next >= target) ? target :
6              static_cast<uint16_t>(next);
7      }
8      if (current > target) {
9          return (current > target + step) ?
10             static_cast<uint16_t>(current - step) : target;
11      }
12  }

```

```

11     return current;
12 }
13 // Appele a chaque tick de controle (20 ms) avec ESC_SLEW_US = 30
   uss/tick
14 outEsc1Us_ = slew(outEsc1Us_, e1t, Calibration::ESC_SLEW_US);

```

La variation maximale autorisée est de 30 μ s par tick de contrôle (20 ms), soit 1500 μ s/s maximum. Cela protège la transmission mécanique contre les changements brusques de consigne.

4.3.5. Armement ESC — sécurité démarrage

Listing 9: ManualController.cpp – procédure d’armement

```

1  if (!escArmed_) {
2      if (frame.ch3 <= Calibration::ESC_ARM_MAX_US) {
3          if (armStartMs_ == 0) armStartMs_ = nowMs;
4          if ((nowMs - armStartMs_) >= Calibration::ESC_ARM_MS) {
5              escArmed_ = true; // Arme apres 2 secondes en dessous
                               du seuil
6          }
7      } else {
8          armStartMs_ = 0; // Reset si les gaz remontent
9      }
10     cmd.esc1Us = Calibration::ESC_STOP_US;
11     return cmd; // Pas de commande moteur avant
                  armement
12 }

```

L’ESC ne peut être armé que si le canal de gaz reste sous le seuil (1300 μ s) pendant 2 secondes consécutives. Tout dépassement remet le compteur à zéro.

5. Phase 1 — Contrôle manuel RC

5.1. Réalisation

La Phase 1 porte sur le contrôle manuel complet par radiocommande. Fichiers implémentés :

Fichier	Rôle
AxpPower.h/.cpp	Init AXP192 : LDO2 (LoRa), LDO3 (GPS), DCDC1 (3,3 V), ADC batterie activé
RcReceiver.h/.cpp	4 canaux, ISR CHANGE, portMUX, timeout 100 ms
McpwmActuators.h/.cpp	MCPWM timer 0 (voile + safran), timer 1 (ESC), slew rate
ModeManager.h/.cpp	CH5 → Failsafe / ManualServo / ManualProp / Automatic
ManualController.h/.cpp	Aileron binaire, safran proportionnel, armement + gaz ESC
DroneApp.h/.cpp	Orchestrateur : ticks 20 ms (contrôle), 200 ms (debug), 1 s (LoRa)
Types.h	ControlMode, RcFrame, ActuatorCommand

5.2. Modes de vol

CH5 (µs)	Mode	Comportement
< 1000	Automatic	Navigation GPS autonome
1400–1600	ManualServo	CH2 → aileron (bascule $\pm 10^\circ$), CH4 → safran (proportionnel)
> 1800	ManualProp	CH3 → gaz ESC (avec armement), CH4 → safran
Signal absent	Failsafe	Tous actionneurs aux positions de sécurité

5.3. Contrôle de l'aileron — logique binaire

L'aileron du drone n'a que deux positions utiles ($+10^\circ$ ou -10°). Le canal CH2 est lu de façon binaire : si le manche dépasse la bande morte ($\pm 35 \mu\text{s}$ autour de $1500 \mu\text{s}$), l'état bascule et reste mémorisé jusqu'au prochain changement :

Listing 10: Logique aileron binaire avec etat memorise

```

1 if (!sailStateInitialized_) {
2     sailState_ = (frame.ch2 >= RC_MID_US) ? +1 : -1;
3     sailStateInitialized_ = true;
4 }
5 if (frame.ch2 > RC_MID_US + db) sailState_ = +1;
```

```

6 else if (frame.ch2 < RC_MID_US - db) sailState_ = -1;
7 // Pas de reset dans la bande morte : l'etat precedent est maintenu
8
9 cmd.sailUs = (sailState_ > 0) ? SAIL_PLUS_US : SAIL_MINUS_US;

```

5.4. Résultats des tests — Phase 1

- [OK] 106 tests unitaires compilés et exécutés nativement sur Linux (sans matériel) — tous passent
- [OK] Compilation arduino-cli : 27 % flash, 7 % RAM
- [OK] Aileron bascule correctement entre 1465 μ s et 1575 μ s
- [OK] Safran répond proportionnellement au manche CH4
- [OK] Armement ESC fonctionnel (2 s sous le seuil)
- [OK] Failsafe déclenché en moins de 100 ms après perte du signal

6. Phase 2 — GPS et navigation

6.1. Réalisation

Fichier	Rôle
GpsUart.h/.cpp	Serial1 GPIO34/12, TinyGPSPlus, capture NMEA, CFG-CFG reset + CFG-ANT au boot
Navigator.h/.cpp	Haversine (distance en m + relèvement en degrés), fonctions stateless dans un namespace
MissionPlan.h	Liste de 16 waypoints max + mode (Linéaire/Circuit)
MissionManager.h/.cpp	Machine à états : Idle → Running → Returning → Complete

6.2. Problème critique — GPS muet après config flash

Symptôme : le compteur `chars` reste bloqué à 10 (un frame UBX ACK-ACK), le GPS est alimenté mais ne produit aucune phrase NMEA.

Cause : une session précédente avec l’IDE Arduino ou u-center avait sauvegardé en flash interne du NEO-6M une configuration CFG-PRT désactivant la sortie NMEA. Cette configuration survit aux coupures d’alimentation et au retrait de la batterie.

Solution : envoyer un reset usine UBX CFG-CFG au démarrage :

Listing 11: GpsUart.cpp – reset usine NEO-6M au démarrage

```

1 // UBX CFG-CFG : efface la config flash, charge les valeurs ROM
2 static const uint8_t cfgReset[] = {
3     0xB5,0x62, 0x06,0x09,          // en-tete UBX + classe/ID CFG-CFG
4     0x0D,0x00,                      // longueur payload = 13 octets
5     0xFF,0xFF,0x00,0x00,          // clearMask = 0xFFFF (effacer
        tout)
6     0x00,0x00,0x00,0x00,          // saveMask = 0x0000
7     0xFF,0xFF,0x00,0x00,          // loadMask = 0xFFFF (charger ROM)
8     0x17,                          // deviceMask =
        SPI+I2C+Flash+EEPROM
9     0x2F,0xAE                      // CRC
10 };
11 Serial1.write(cfgReset, sizeof(cfgReset));
12 delay(500);

```

6.3. Problème critique — Antenne active non alimentée

Symptôme : NMEA reçu (`chars` s’incrémente), mais `visible = 0` satellites même en extérieur sous ciel dégagé, avec antenne externe connectée.

Cause : le superviseur d’antenne du NEO-6M est désactivé par défaut. Sans l’activer :

1. Aucune tension de polarisation n’est fournie au LNA de l’antenne active Taoglas : le LNA est sans alimentation.
2. Le commutateur RF du T-Beam (`ANT_FLAG`) reste sur l’antenne céramique interne : le connecteur u.FL est court-circuité.

Listing 12: GpsUart.cpp – activation superviseur d’antenne (CFG-ANT)

```

1 // UBX CFG-ANT : flags = 0x001B active le superviseur d'antenne

```

```

2 // (svcs=1: tension RF, scd=1: detection court-circuit,
3 // pdwnOnSCD=1: coupure sur court-circuit, recovery=1)
4 static const uint8_t cfgAnt[] = {
5     0xB5,0x62, 0x06,0x13,
6     0x04,0x00,
7     0x1B,0x00,          // flags = 0x001B
8     0x00,0x00,          // pins = 0 (valeur par défaut)
9     0x38,0x43
10 };
11 Serial1.write(cfgAnt, sizeof(cfgAnt));
12 delay(200);

```

Résultat après correction : fix acquis en moins de 60 s en extérieur, 7 satellites visibles, HDOP = 2,2, position confirmée (zone Brest, 48.360476° N / -4.566822° W).

6.4. Note sur le démarrage à froid (cold start)

Sans batterie LiPo connectée, chaque mise sous tension perd l'almanach GPS stocké. Le démarrage à froid prend environ 2 minutes en extérieur dégagé. Avec la batterie, les éphémérides sont conservés et le fix s'obtient en quelques secondes (démarrage chaud). La LED TIMEPULSE sur le NEO-6M clignote à 1 Hz quand un fix est acquis.

7. Phase 3 — LoRa et télémétrie

7.1. Réalisation

Fichier	Rôle
LoRaRadio.h/.cpp	Driver SX1276, SPI HSPI, 433 MHz, TX bloquant, RX par polling
LoRaComm.h/.cpp	Heartbeat JSON TX à 1 Hz, dispatcher de commandes (navigate, stop, home, waypoints, wind, restart)
transceiver.ino	Sketch station sol : CLI compacte + passerelle JSON bidirectionnelle

7.2. Protocole JSON LoRa

Listing 13: Heartbeat drone vers station sol (période 1 s)

```
{
  "origin": "boat",
  "type": "info",
  "message": {
    "mode": "navigate",
    "location": [48.360476, -4.566822],
    "servos": {"sail": 0, "rudder": 0},
    "control_mode": "autonomous",
    "heading": 270.5,
    "wind": 180,
    "bat": 7.42,
    "waypoints": {"total": 3, "current": 1}
  }
}
```

Listing 14: Commandes station sol vers drone

```
{"origin":"server","type":"command","message":{"navigate":true}}
{"origin":"server","type":"command","message":{"stop":true}}
{"origin":"server","type":"command","message":{"restart":true}}
{"origin":"server","type":"command","message":
  {"waypoints":{"number":2,"points":["48.36",-4.56,48.37,-4.55]}}}
{"origin":"server","type":"command","message":
  {"home":{"lat":48.36,"lon":-4.56}}}
```

7.3. Résultats

- [OK] TX confirmé (compteur tx s'incrémente à 1 Hz)
- [OK] 7 commandes testées via script Python (`lora_uplink_test.py`)
- [OK] RSSI : -101 à -106 dBm à 5 cm de distance (valeur attendue)
- [OK] GPS et LoRa opèrent simultanément sans interférence
- [] Test de portée réelle en extérieur (prévu Phase 7)

`LoRa.endPacket()` est synchrone. À SF7/BW125 kHz, un paquet de ≈ 210 octets nécessite ≈ 450 ms de transmission, bloquant la boucle principale une fois par seconde. Acceptable en Phase 3 ; à passer en mode asynchrone avant la Phase 6 si le jitter du loop de contrôle devient critique.

8. PCB Version 2 — Améliorations matérielles

8.1. Motivations

La Version 2 du PCB corrige tous les problèmes identifiés sur la V1 et intègre les retours d'expérience du développement logiciel :

1. Déplacement des broches RC vers des GPIO sans conflits
2. Amplification de niveau logique 3,3 V $\rightarrow$ 5 V par transistors NPN BC548
3. Diviseur résistif haute impédance pour la mesure de tension batterie
4. Libération de GPIO21/22 pour une utilisation I2C future
5. Suppression du CH6 (libère GPIO23 pour le RST LoRa)
6. Un seul ESC (suppression ESC2)

8.2. Correspondance des broches — PCB V2

Signal	GPIO	Notes
RC CH2 (aileron)	39	SVN, entrée seule, pas de drain continu
RC CH3 (gaz)	14	Libéré (n'est plus partagé avec I2C)
RC CH4 (safran)	13	Libéré
RC CH5 (mode)	4	Commutateur 3 positions
Servo voile (PWM1)	2	Via transistor BC548
Servo safran (PWM2)	25	Via transistor BC548
ESC1	15	Via transistor BC548
ADC batterie	36	SVP, ADC1_CH0, entrée seule
LoRa SCK	5	SPI HSPI
LoRa MISO	19	SPI HSPI
LoRa MOSI	27	SPI HSPI

Signal	GPIO	Notes
LoRa CS	18	Actif bas
LoRa RST	23	Maintenant libre (CH6 supprimé)
LoRa IRQ (DIO0)	26	
GPS RX	34	Entrée seule
GPS TX	12	
I2C SDA	21	AXP192 interne + connecteur d'extension
I2C SCL	22	AXP192 interne + connecteur d'extension

8.3. Amplification de niveau logique — Transistors BC548

Les sorties PWM de l'ESP32 sont à 3,3 V. Les servos et ESC Pro-Tronik attendent 5 V. La V2 utilise des transistors NPN BC548 en montage émetteur commun avec une résistance de base 5,1 k Ω et une résistance de collecteur 11,3 k Ω :

Initialement, le canal OPR_B (safran) utilisait MCPWM_DUTY_MODE_1 (inversion), sous l'hypothèse que le transistor NPN inversait le signal et qu'il fallait pré-inverser. En réalité, **les deux canaux** (OPR_A et OPR_B) fonctionnent correctement avec DUTY_MODE_0 à travers le circuit BC548. L'utilisation de DUTY_MODE_1 sur OPR_B produisait une double inversion, rendant le signal inutilisable pour le servo safran. La correction consiste à utiliser DUTY_MODE_0 (actif haut) pour les deux canaux.

8.4. Mesure de tension batterie — diviseur haute impédance

Listing 15: BatteryAdc.cpp – calcul de tension

```

1 // R5 = 560 kOhm (cote haut, batterie vers ADC)
2 // R6 = 120 kOhm (cote bas, vers GND)
3 // V_bat = V_adc x (R5 + R6) / R6 = V_adc x 5,667
4 static constexpr float DIVIDER_RATIO =
5     (560000.0f + 120000.0f) / 120000.0f; // = 5,667
6
7 void BatteryAdc::begin() {
8     analogSetPinAttenuation(BoardConfig::BATTERY_ADC_PIN, ADC_11db);
9 }

```

```
10 float BatteryAdc::readVolts() {  
11     uint32_t vPin_mV =  
        analogReadMilliVolts(BoardConfig::BATTERY_ADC_PIN);  
12     return (vPin_mV * DIVIDER_RATIO) / 1000.0f;  
13 }
```

Avec des résistances de 560 k Ω et 120 k Ω , la consommation du diviseur à pleine charge (7,4 V) est de $7,4/680000 \approx 10,9 \mu\text{A}$ — négligeable.

8.5. Problème rencontré — résistances inversées

Symptôme : lecture ADC affichant 17,9 V au lieu de ≈ 5 V.

Cause : les résistances physiques étaient câblées à l'inverse : 120 k Ω côté haut et 562 k Ω côté bas. Le point de mesure ADC se retrouvait à $5,0 \times 562 / (120 + 562) \approx 4,12$ V, saturant l'ADC. La valeur lue était multipliée par le ratio inversé $682/120 = 5,683$, d'où 17,9 V.

Solution : inversion physique des résistances. Le code était correct.

8.6. Gestion de l'AXP192 — I2C persistant

Dans la V1 du code, `Wire.end()` était appelé à la fin de l'init AXP192 pour libérer GPIO21/22 destinés au RC. Dans la V2, ces broches sont libres : GPIO21 et GPIO22 restent dédiés à l'I2C. `Wire.end()` est supprimé du chemin de succès ; le bus I2C reste actif pour de futurs capteurs via un connecteur d'extension.

9. Problèmes rencontrés et solutions détaillées

Cette section recense tous les problèmes significatifs rencontrés au cours du développement, classés par domaine.

9.1. Matériel — ESP32 / T-Beam

9.1.1. GPIO35 — drain de 20 mA continu

Le T-Beam V1.1 comporte un diviseur résistif d'impédance totale $\approx 370 \Omega$ permanentement connecté à GPIO35. Ce diviseur consomme environ 20 mA en continu depuis la batterie, soit $\approx 1,8 \text{ W}$, indépendamment de toute lecture ADC.

Décision : ne jamais lire GPIO35. L'ADC batterie est implémenté sur GPIO36 (SVP, ADC1, entrée seule, sans drain) avec diviseur externe haute impédance ($560 \text{ k}\Omega + 120 \text{ k}\Omega$).

9.1.2. LoRa RST — pin incorrect dans la documentation

La documentation de référence de LilyGO et divers exemples en ligne indiquent GPIO14 pour le RESET du SX1276. Le hardware T-Beam V1.1 câble physiquement le RESET sur GPIO23.

Le code Post-Claude utilise `LORA_RST_PIN = 23` (correct). Dans la PCBv3/main actuelle, GPIO23 est libre (CH6 supprimé), donc le RESET matériel est maintenant fonctionnel.

9.1.3. AXP192 — GPS non alimenté

Le GPS (u-blox NEO-6M) est alimenté par le rail LDO3 de l'AXP192. Sans activer ce rail en code, le GPS ne reçoit aucune alimentation et reste silencieux, même si le module est physiquement présent.

`AxpPower::begin()` active explicitement LDO3 à 3300 mV avant toute initialisation du GPS. LDO2 (LoRa) et DCDC1 (3,3 V) sont également activés.

9.2. Logiciel — Firmware

9.2.1. DUTY_MODE_1 sur OPR_B — servo safran inopérant

Symptôme : le servo safran (Regatta ECO II, GPIO25) ne répondait à aucune commande. Le servo aileron (GPIO2) fonctionnait correctement. Les valeurs de consigne semblaient correctes dans les logs.

Cause : le code utilisait MCPWM_DUTY_MODE_1 (signal inversé) sur OPR_B (safran), supposant que le transistor NPN BC548 inverserait le signal et que la pré-inversion était nécessaire. En réalité, le circuit BC548 en émetteur commun produit une inversion de son côté ; appliquer une inversion côté MCPWM entraîne une double inversion, résultant en un signal inutilisable (19000 μ s de haut, 1000 μ s de bas au lieu de l'inverse).

Comportement observé : signal en sortie du transistor de fréquence 50 Hz mais avec un rapport cyclique inverse : le servo interprète le temps bas comme la commande, ce qui sort de la plage valide.

Suppression de l'appel `mcpwm_set_duty_type()` pour OPR_B : les deux canaux utilisent maintenant DUTY_MODE_0 (actif haut) sans inversion. Le servo safran répond immédiatement.

9.2.2. Seuil d'armement ESC trop bas

Symptôme : l'ESC ne s'arme jamais malgré la procédure correcte. Les logs indiquent : CH3=1265 alors que ESC_ARM_MAX_US=1050.

Cause : l'émetteur Pro-Tronik PTR-6A n'atteint pas 1000 μ s à la position minimale de son manche de gaz : le minimum réel mesuré est $\approx$ 1265 μ s. Le seuil d'armement de 1050 μ s n'est donc jamais atteint physiquement.

Le seuil ESC_ARM_MAX_US est porté de 1050 à 1300 μ s dans `Calibration.h`, permettant l'armement au minimum de manche réel tout en conservant une marge de sécurité suffisante.

9.2.3. Mode bloqué en Automatic — confusion CH4/CH5

Symptôme : après mise sous tension, le drone restait systématiquement en mode Automatic, même avec le commutateur de mode en position ManualServo.

Cause : dans la version initiale, ModeManager lisait CH4 pour déterminer le mode de vol, alors que le commutateur 3 positions physique est câblé sur CH5. Le canal CH4 (manche analogique) renvoyait une valeur $< 1000 \mu s$ au repos, déclenchant le mode Automatic.

Remplacement de toutes les références `frame.ch4` par `frame.ch5` dans `ModeManager.cpp`, et renommage des constantes `CH4_*` en `CH5_*` dans `BoardConfig.h`.

9.2.4. GPS — config flash désactivant NMEA

Voir Section 6.2 pour la description complète et la solution.

9.2.5. GPS — antenne active non alimentée

Voir Section 6.3 pour la description complète et la solution.

9.2.6. Port série occupé lors du flash

Symptôme : échec du transfert firmware avec l'erreur `could not open port /dev/ttyUSB0: [Errno 16] Device or resource busy`.

Cause : un processus (moniteur série ou script Python) tenait le port série ouvert pendant la tentative de flash.

Identifier et tuer le processus : `fuser /dev/ttyUSB0` puis `kill <PID>`, ou simplement fermer le moniteur série avant le flash.

10. État actuel du système

10.1. Résumé des phases

Phase	Objectif	État	Notes
1	Contrôle manuel RC	[OK] Complet	106 tests, testé en vol
2	GPS + navigation	[OK] Complet	Fix testé en extérieur
3	LoRa + télémétrie	[OK] Complet	TX/RX vérifiés
PCB V2	Nouveau câblage + I2C libre	[OK] Complet	Compilé et testé
4	Capteurs I2C	[] À faire	GPIO21/22 disponibles
5	Estimateur de vent	[] À faire	30 m + 2 amures
6	Navigation autonome	[] À faire	SailAutoMode
7	Mission complète	[] À faire	Waypoints + LoRa

10.2. Utilisation flash / RAM

Sketch	Flash	RAM
main/main.ino (drone)	27 %	7 %
transceiver/transceiver.ino	23 %	6 %

10.3. Valeurs de calibration actuelles

Paramètre	Valeur	Notes
SAIL_CENTER_US	1520 µs	Centre Futaba S3003 (non 1500!)
SAIL_PLUS_US	1575 µs	+10° — à vérifier sur banc
SAIL_MINUS_US	1465 µs	−10° — à vérifier sur banc
ROTOR_CENTER_US	1500 µs	Safran centré (position de repos)
ROTOR_MIN_US	1000 µs	Butée basse
ROTOR_MAX_US	2000 µs	Butée haute
ESC_STOP_US	1000 µs	Moteur arrêté
ESC_MAX_US	2000 µs	Plein régime

Paramètre	Valeur	Notes
ESC_ARM_MAX_US	1300 μ s	Seuil d'armement
ESC_ARM_MS	2000 ms	Durée maintien pour armer
ESC_SLEW_US	30 μ s/tick	Limitation variation ESC
RC_DEADBAND_US	35 μ s	Bande morte manche
Ratio diviseur batterie	5,667	(560k + 120k) / 120k

11. Plan de développement futur

11.1. Phase 4 — Capteurs I2C

Maintenant que GPIO21 et GPIO22 sont libres (bus I2C persistant, plus de `Wire.end()`), des capteurs peuvent être connectés au connecteur d'extension du PCB V2. Travaux à effectuer :

- Identifier les capteurs requis et leurs adresses I2C
- Implémenter `SensorBus.cpp` avec scan de bus et collecte d'échantillons
- Implémenter `SensorManager` (polling à 500 ms, buffering)

11.2. Phase 5 — Estimateur de vent

Sans capteur de vent, la direction est inférée depuis le track GPS :

1. Collecter le cap GPS sur deux amures différentes (état aileron ± 1)
2. Le vent bissecte l'angle entre les deux caps
3. Minimum 30 m parcourus sur chaque amure pour valider l'estimation
4. Possibilité de forcer la direction via commande LoRa (`wind-command`)

11.3. Phase 6 — Navigation autonome à la voile

Listing 16: Algorithme SailAutoMode (pseudo-code)

```

1 // Entrees : position GPS, waypoint, cap GPS, direction vent estimee
2 // Sorties : sailUs (+/- SAIL_DELTA), rotorUs (position continue)
3 float bearing = Navigator::bearingDeg(pos, waypoint);
4 float relWind = normalizeAngle(bearing - windDir);

```

```

5 if (abs(relWind) < UPWIND_ZONE_DEG) {
6     // Waypoint dans la zone morte (vent debout) -- virement de bord
7     // Calculer la route de louvoyage optimale
8 } else {
9     // Route directe possible -- choisir l'amure et regler le cap
10    sailState = chooseTack(relWind);
11    cmd.sailUs = sailState > 0 ? SAIL_PLUS_US : SAIL_MINUS_US;
12    cmd.rotorUs = steerToward(pos, waypoint, heading);
13 }

```

Un **WinchTracker** logiciel sera nécessaire : le Regatta ECO II n'a pas de retour de position. Il faudra intégrer vitesse $\times$ temps pour estimer la position du treuil et détecter les butées mécaniques.

11.4. Phase 7 — Mission complète

- Chargement des waypoints par LoRa depuis la station sol
- Démarrage de la mission, monitoring par heartbeat LoRa
- Journalisation des données capteurs sur SPIFFS
- Test de portée LoRa en conditions réelles

12. Environnement de développement

12.1. Outils

Outil	Notes
Arduino IDE 2.3.8	AppImage Linux, écriture code + upload + moniteur série
arduino-cli v1.4.1	Compilation en ligne de commande (CI)
KiCad 9.0	Conception PCB V1 et V2
pdflatex	Compilation de ce rapport
ESP32 Arduino Core 3.x	Bibliothèque cible

Outil	Notes
AXP202X_Library 1.1.2	Gestion AXP192
TinyGPSPlus 1.0.3	Parseur NMEA
LoRa 0.8.0	Driver SX1276

12.2. Commandes utiles

Listing 17: Compiler et flasher le firmware

```
# Compilation
~/bin/arduino-cli compile --fqbn esp32:esp32:t-beam \
  /chemin/vers/Post-Claude/main

# Flash (remplacer /dev/ttyUSB0 selon le systeme)
~/bin/arduino-cli upload --fqbn esp32:esp32:t-beam \
  --port /dev/ttyUSB0 /chemin/vers/Post-Claude/main

# Surveiller le port serie
python3 -m serial.tools.miniterm /dev/ttyUSB0 115200
```

13. Journal de développement

Date	Activité
2026-04-27	Lecture du projet, analyse de l'architecture, création de CLAUDE.md. Recherche composants (Futaba S3003, Regatta ECO II, Pro-Tronik, T-Beam V1.1). Phase 1 codée (10 fichiers). Diagnostic drain GPIO35 (≈ 20 mA).
2026-04-27	Tests unitaires natifs (59 tests). Bug ESC : <code>disarm()</code> manquant lors du passage en mode ManualServo — corrigé. Tous les tests passent.
2026-04-28	47 nouveaux tests (106 au total). Correction structure Arduino IDE (<code>src/</code> requis). Compilation arduino-cli : 24 % flash, 7 % RAM.
2026-04-28	Phase 2 — GPS : GpsUart, Navigator (haversine), MissionPlan, MissionManager. Diagnostic GPS froid en extérieur (sans LiPo = cold start).

Date	Activité
2026-05-04	Phase 3 — LoRa : driver, LoRaComm (JSON), transceiver sol. TX vérifié. 7 commandes testées via Python. Phase 3 complète.
2026-05-20	Correction MCPWM DUTY_MODE_1 → DUTY_MODE_0 sur OPR_B (safran). Diagnostic et correction GPS (CFG-CFG + CFG-ANT). Fix GPS obtenu en extérieur.
2026-05-21	Analyse schéma PCB V2 (KiCad). Validation des nouvelles connexions (BC548, diviseur, GPIO libres).
2026-05-22	Refactoring PCBv3 (sketch test) : suppression ESC2, CH5 pour mode, CH4 pour safran. Correction seuil armement (1050 → 1300 μ s). Compilation, flash, tests physiques. Aileron et safran fonctionnels. Armement ESC validé.
2026-05-24	Mise à jour du sketch production (<code>main/</code>) pour le nouveau câblage PCB V2 : GPIO39/14/13/4 pour RC, GPIO15 pour ESC, GPIO36 pour batterie, LoRa RST=23, Wire.end() supprimé. Compilation propre (27 % flash, 7 % RAM).

A. Spécifications des composants

A.1. Futaba S3003 — Servo aileron

Paramètre	Valeur
Fréquence PWM	50 Hz
Plage d'impulsions	1000–2000 μ s
Centre (neutre)	1520 μs (standard Futaba, pas 1500 !)
Bande morte	$\approx$ 100 μ s
Tension d'alimentation	4,8 V ou 6,0 V
Couple (4,8 V / 6 V)	3,2 kg · cm / 4,1 kg · cm
Vitesse (4,8 V / 6 V)	0,23 s/60° / 0,19 s/60°
Dimensions	40,4 × 19,8 × 36 mm

Paramètre	Valeur
Masse	37,2 g

A.2. Graupner Regatta ECO II (mod. 5176) — Treuil safran

Paramètre	Valeur
Type	Treuil multi-tour positionnel
Fréquence PWM	50 Hz
Plage d'impulsions	1000–2000 μ s
Centre (repos)	1500 μ s (tient la position)
Course mécanique	6 tours (± 3 tours depuis le centre)
Tension	4,8–7,2 V
Courant repos / charge	300–350 mA / 2500–3300 mA
Couple (6 V / 7,4 V)	$\approx 9,5$ kg · cm / $\approx 10,9$ kg · cm
Dimensions	40,6 × 20 × 38,9 mm
Masse	56 g

A.3. Pro-Tronik PTR-6A / R8X

Paramètre	Valeur
Protocole	FHSS 2,4 GHz (propriétaire Pro-Tronik)
Canaux TX	6 (4 proportionnels + CH5 3 positions + CH6 2 positions)
Canaux RX	8 (R8X)
PWM neutre	1500 μ s
Plage PWM	1000–2000 μ s
Fréquence PWM	50 Hz
Portée	≈ 2400 m
Minimum manche gaz (mesuré)	≈ 1265 μ s (pas 1000 μ s !)